

ADT

Table of contents

First-Class ADT Pattern	3
First-Class ADT Pattern	3
Encapsulation	3
Abstract Data Type (ADT)	3
ADT in C e in Python	3
First-Class ADT	4
ADT: teoria vs C (con e senza incapsulamento) vs Python	5
STACK	5
Operazioni essenziali	7
Operazioni complementari	7
Operazioni di gestione (tipiche in C)	7
Implementazione dello Stack con Array Statico	7
Scelte progettuali	8
Responsabilità del <code>main</code>	8
Dimensione massima dello stack	9
Assenza di incapsulamento completo	9
Uso dei puntatori	9
Altre note	10
Esempio di <code>main</code>	10
<code>module.h</code>	11
<code>module.c</code>	13
Discussione e miglioramenti	16
Verifica condizioni di overflow/underflow	16
Stack con capacità dinamica	16
Dangling pointer	18
Stack con Array Statico, incapsulamento completo	19
<code>module.h</code>	19
<code>module.c</code>	20
Implementazione dello Stack con Lista Concatenata	27
Componenti principali	27
Interfaccia	30
Nota su <code>const</code>	30

Esempio di main	31
Implementazione delle funzioni fondamentali	31
initStack	32
isEmpty	32
push	32
pop	34
peek	36
deleteStack	36
Stack con Lista Concatenata, incapsulamento completo	38
Soluzione	39
Applicazioni dello stack - esercizi	41
QUEUE	43
Operazioni fondamentali	44
Osservazioni	45
Implementazione della Queue mediante array	45
Array circolare	46
Stato della coda	48
Funzioni principali	49
initializeQueue	49
isEmpty	49
isFull	50
enqueue	50
dequeue	51
deleteQueue	51
Esempio	52
Osservazioni	52
Note su capacity	52
Implementazione della Queue mediante lista concatenata	53
Interfaccia	56
Strutture dati interne	56
createNewNode	57
initializeQueue	57
isEmpty	58
isFull	58
enqueue	58
dequeue	59
deleteQueue	60
Funzione di debug: show	61
Vantaggi e svantaggi della linked list	61
Vantaggi	61
Svantaggi	62
Considerazioni finali	62
Esercizi proposti	63

FONTI PRINCIPALI

64

First-Class ADT Pattern

First-Class ADT Pattern

Encapsulation

L'**incapsulamento** (**encapsulation**) è uno strumento fondamentale per gestire la complessità e separare le responsabilità all'interno di un sistema.

Quando si progetta software, è utile distinguere tra due prospettive:

- **Comportamento esterno (interfaccia):** come un componente viene utilizzato (come interagisce con l'esterno)
 - **Implementazione interna:** come il componente è effettivamente costruito (come è strutturato internamente)
-

Abstract Data Type (ADT)

Un **Tipo di Dato Astratto (ADT)** definisce:

- un **insieme di valori**
- un **insieme di operazioni** su tali valori

senza specificare come questi valori siano rappresentati internamente.

https://en.wikipedia.org/wiki/Abstract_data_type

ADT in C e in Python

Nel linguaggio C, l'incapsulamento di un ADT si realizza tipicamente tramite:

- la dichiarazione di un **tipo incompleto** (*incomplete type*, dato opaco) nel file header `.h`
- la manipolazione dei valori esclusivamente tramite **puntatori**
- la definizione completa della struttura nascosta nel file `.c`

Il tipo è detto *incompleto* perché la sua struttura interna non è visibile al codice che utilizza l'ADT.

In **Python**, gli ADT si implementano tipicamente tramite **classi**, che nascondono (parzialmente) i dettagli interni e forniscono un'interfaccia pubblica.

First-Class ADT

Un ADT è detto **first-class** se i suoi valori possono essere trattati come qualsiasi altro valore del linguaggio.

https://en.wikipedia.org/wiki/First-class_citizen

In C, questo significa che (tipicamente tramite puntatori) un ADT può:

- essere **passato come argomento a una funzione**
- essere **restituito da una funzione**
- essere **assegnato a una variabile**
- essere **memorizzato in strutture dati**

In C, gli ADT sono generalmente gestiti tramite puntatori: l'assegnazione tra variabili copia il puntatore (riferimento), non il contenuto dell'oggetto.

Nel contesto della programmazione, si parla di “**cittadini di prima classe**” (**first-class citizens**) per indicare entità che possono essere usate liberamente: assegnate a variabili, passate come argomenti, restituite da funzioni e memorizzate in strutture dati. In **C**, sono first-class i valori “semplici” (int, float, puntatori, ecc.), mentre le **funzioni non lo sono pienamente**: non possono essere assegnate direttamente a variabili, ma solo tramite **puntatori a funzione**. In **Python**, invece, anche le funzioni sono veri cittadini di prima classe.

```
// Esempio in C
int add(int a, int b) { return a + b; }

int main() {
    int (*f)(int, int) = add; // puntatore a funzione
    int r = f(2, 3);          // uso indiretto
}
```

```
def f1(x):
    print(x)
    #return None
def f2(x):
    print(x)

a = f2 # assegnazione diretta

f1(a) # <function f2 at 0x110211ee0>
```

- **ADT con incapsulamento completo dei dati.**
L'attenzione è su **astrazione** e **incapsulamento**: la rappresentazione interna è nascosta e accessibile solo tramite le operazioni definite.
(È un'implementazione **corretta e robusta** di un ADT).
- **ADT senza incapsulamento completo dei dati.**
L'attenzione è solo sull'**astrazione**: la struttura interna è visibile o direttamente accessibile.
(È un'implementazione **debole** di un ADT, che viola l'incapsulamento).
- Un ADT può inoltre essere *first-class* se i suoi valori possono essere passati, restituiti e assegnati come qualsiasi altro valore del linguaggio.

ADT: teoria vs C (con e senza incapsulamento) vs Python

Aspetto	Teoria (ADT)	C (con incapsulamento)	C (senza incapsulamento)	Python
Definizione	Insieme di valori + operazioni	Uguale alla teoria	Uguale alla teoria	Uguale alla teoria
Rappresentazione interna	Non specificata	Nascosta (<code>struct</code> in <code>.c</code>)	Visibile (<code>struct</code> in <code>.h</code>)	Attributi della classe
Incapsulamento	Non richiesto	Forte (tipo incompleto)	Assente	Debole (convenzioni)
Accesso ai dati	Tramite operazioni	Solo tramite funzioni	Diretto + funzioni	Diretto + metodi
Modifica dei dati	Tramite operazioni	Tramite funzioni	Diretto + funzioni	Diretto + metodi
First-class	Dipende dal linguaggio	Tramite puntatori	Tramite puntatori	Nativamente

STACK

[https://en.wikipedia.org/wiki/Stack_\(abstract_data_type\)](https://en.wikipedia.org/wiki/Stack_(abstract_data_type))

Il codice utilizzato è disponibile nel repository alla cartella `src/ADT/stack_*`

Lo **stack** (o **pila**) è un ADT in cui gli elementi vengono gestiti secondo la politica **LIFO** (*Last In, First Out*): l'ultimo elemento inserito è il primo a uscire.

Le operazioni fondamentali sono:

- **push**: inserisce un nuovo elemento in cima allo stack

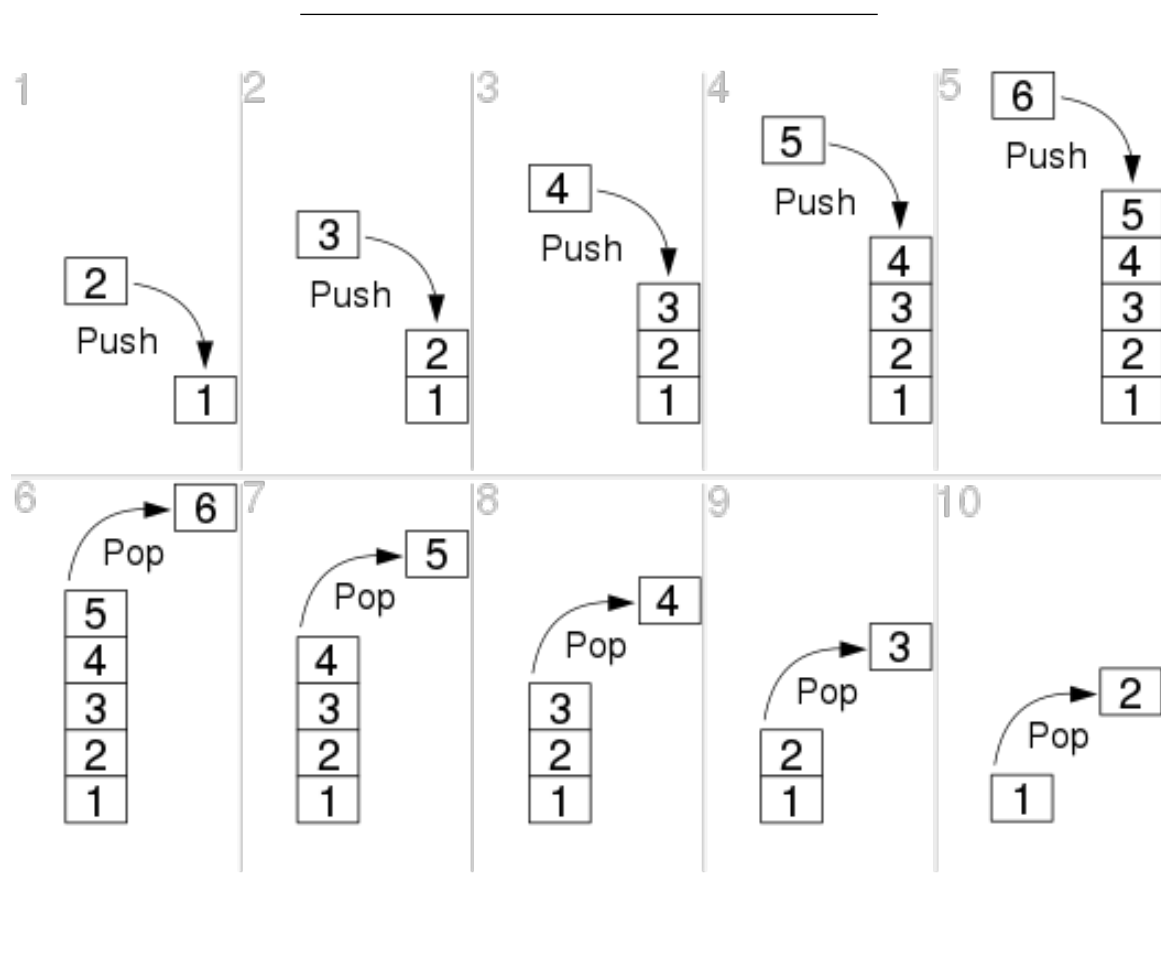
- **pop**: rimuove l'elemento in cima allo stack
- **peek**: restituisce l'elemento in cima senza rimuoverlo

Tutte queste operazioni agiscono su un'unica estremità della struttura, detta **cima** (*top*).

In C, lo stack può essere implementato con:

- un **array**
- una **lista concatenata**

La scelta dell'implementazione influenza gli aspetti pratici (uso della memoria, dimensione massima, efficienza), ma non modifica il comportamento astratto dello stack. Uno stack è definito dalle sue operazioni e dal suo comportamento LIFO, non dal modo in cui viene implementato.



Operazioni essenziali

Sono le operazioni **minime** che definiscono il comportamento dello stack:

- `push(Stack* s, T x)`
Inserisce un elemento in cima allo stack
- `pop(Stack* s)`
Rimuove l'elemento in cima allo stack

Operazioni complementari

Non sono strettamente necessarie per definire l'ADT, ma sono **molto utili** nella pratica:

- `peek(Stack* s) / peek(...)`
Restituisce l'elemento in cima senza rimuoverlo
- `isEmpty(Stack* s)`
Verifica se lo stack è vuoto
- `size(Stack* s)`
Restituisce il numero di elementi

Operazioni di gestione (tipiche in C)

In C sono fondamentali per gestire memoria e ciclo di vita:

- `create() / init()`
Crea e inizializza uno stack
- `destroy(Stack* s)`
Libera la memoria associata
- `isFull(Stack* s)`
non è un'operazione propria dello stack come ADT, ma può essere introdotta in implementazioni con capacità limitata.

Implementazione dello Stack con Array Statico

Uno stack può essere implementato utilizzando:

- un **array statico** per memorizzare gli elementi
- una variabile intera **top** che indica la posizione dell'elemento in cima allo stack

```
typedef struct stack {  
    int stackArray[N];  
    int top;  
} Stack;
```

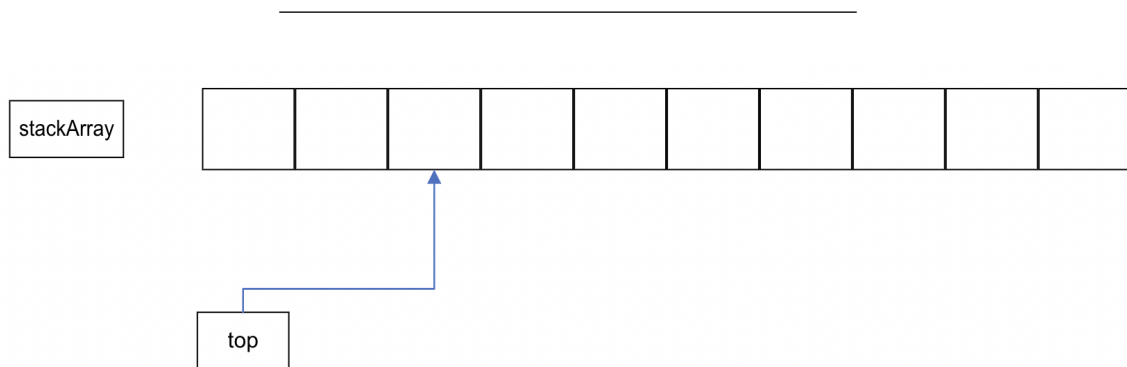
Quando lo stack è vuoto, `top` vale:

```
top = -1;
```

Le operazioni di inserimento o estrazione modificano `top`:

- `push` incrementa `top` e inserisce un elemento
- `pop` restituisce l'elemento in cima e decrementa `top`

In questo modo, `top` rappresenta sempre l'**indice dell'ultimo elemento** inserito nello stack.



Scelte progettuali

Questa versione è **semplificata e intenzionalmente non completa** dal punto di vista progettuale.

Responsabilità del main

In questa implementazione, il controllo delle condizioni di corretto utilizzo dello stack è delegato al programma chiamante (`main`).

In particolare:

- prima di eseguire una `pop` o `peek`, il `main` deve verificare che lo stack **non sia vuoto** mediante la funzione `isEmpty()`

- prima di eseguire una **push**, il **main** deve verificare che lo stack **non sia pieno** mediante la funzione `isFull()`

Questa è una **scelta didattica**:

le funzioni del modulo non effettuano controlli interni e richiedono il rispetto di precise **precondizioni**.

Dimensione massima dello stack

La dimensione dello stack è fissata a compile-time tramite una direttiva:

```
#define STACK_MAX_SIZE 100
```

Questa costante è definita nel file di interfaccia (`.h`) ed è quindi **visibile a tutto il programma**. Si tratta di una scelta che privilegia semplicità e chiarezza del modello a scapito della flessibilità.

Assenza di incapsulamento completo

La struttura è visibile nell'interfaccia `.h`. Il `main` può bypassare le funzioni di interfaccia.

```
s->stackArray[0]=10;  
s->top=0;
```

Uso dei puntatori

Le funzioni operano su **puntatori a Stack**. Questa scelta è motivata da **efficienza**, perché si evita la copia della struttura, e **coerenza progettuale**, perché facilita l'evoluzione verso implementazioni più avanzate, per esempio incapsulamento completo. Tuttavia questo non è un obbligo. Dato che la struttura è visibile nell'interfaccia, il `main` può dichiarare direttamente variabili di tipo `Stack` e manipolare i campi.

Altre note

Questa implementazione presenta alcune limitazioni intenzionali:

- non verifica condizioni di errore (overflow/underflow) nelle funzioni `push()`, `pop()`, `peek()`. Il controllo è demandato al main, che prima di chiamare `push()` deve verificare, mediante `isFull()`, che lo stack abbia ancora capienza, e prima di chiamare `pop()` e `peek()` deve verificare, mediante `isEmpty()`, che lo stack non sia vuoto.
- espone direttamente la struttura dati
- utilizza una dimensione fissa per l'array che memorizza i dati dello stack

Tali scelte la rendono semplice da comprendere e adatta a una prima introduzione, ma spesso non adeguata a contesti reali. Viene fornito un main con un menu ed un test minimali, e una funzione `show()` che mostra tutti gli elementi dello stack.

Le funzioni di debug come `show()` e `automaticTest()` sono **solo per debug**. Una volta terminato il debug dovrebbero essere rimosse dall'interfaccia (o rimosse del tutto), perché non fanno parte dell'ADT, e dichiarate `static` nel `modulo.c`. Rimuovere la dichiarazione di una funzione dal file `.h` è generalmente sufficiente per impedirne l'uso da `main.c`, perché il compilatore non ne vede la dichiarazione. Tuttavia, questo non è completamente sicuro: se qualcuno dichiara manualmente `void show(Stack *);` in `main.c`, il programma può comunque compilare e linkare, perché la funzione esiste in `module.c`. Per evitare questo "bypass", è buona pratica dichiarare `show()` e `automaticTest()` come `static` nel `.c`: in questo modo la funzione è visibile solo all'interno del file e non può essere usata dall'esterno.

Esempio di main

(Nel repository github il main completo)

```
int main() {
    int value = 0;
    Stack *s = initStack();

    // Push (with check)
    if (!isFull(s)) {
        push(s, value);
    } else {
        printf("Stack pieno!\n");
    }

    // Peek (with check)
    if (!isEmpty(s)) {
        value = peek(s);
    } else {
        printf("Stack vuoto!\n");
    }
}
```

```

    // Pop (with check)
    if (!isEmpty(s)) {
        value = pop(s);
    } else {
        printf("Stack vuoto!\n");
    }

    // ora lo stack è vuoto

    push(s, 1);
    push(s, 2);
    push(s, 3);

    show(s);
}

```

module.h

```

// ADT/stack_array_noencaps/module.h

/* STACK
 * Implementazione: array statico
 * SENZA incapsulamento completo
 *
 * È possibile aggirare l'interfaccia e violare l'incapsulamento:
 * s->stackArray[0] = 10;
 * s->top = 0;
 *
 * L'implementazione utilizza un array contenuto in una struttura.
 * L'allocazione della struttura è dinamica, ma questo è solo un dettaglio implementativo.
 * Modificando initStack si potrebbe ottenere un'implementazione completamente statica.
 */

#ifndef MODULE_H
#define MODULE_H
#include <stdbool.h>

/*
 * il .h deve includere altri .h solo se strettamente necessario.
 */

```

```

* La libreria <stdbool.h> definisce
*   il tipo di dato booleano bool
*   le costanti true (1) e false (0).
* Viene usato dall'interfaccia, quindi è necessario includerlo
*/

/* STACK_MAX_SIZE -> capacity of the Static Stack
* STACK_MAX_SIZE is part of the interface
* and it is visible from main()
*/
#define STACK_MAX_SIZE 1000

typedef struct stack {
    int stackArray[STACK_MAX_SIZE];
    int top;
}Stack;

// Function prototypes

// Push element to the top of the stack
// PRECONDIZIONE: stack NON pieno
void push(Stack * stack, int val);

// Remove and return the top most element of the stack
// PRECONDIZIONE: stack NON vuoto
int pop(Stack * stack);

// Return the top most element of the stack
// PRECONDIZIONE: stack NON vuoto
int peek(Stack * stack);

// Check if the stack is in Underflow state or not
bool isEmpty(Stack * stack);

Stack * initStack(); //INIT
void deleteStack(Stack * stack); // Free memory

// Check if the stack is in Overflow state or not
// ONLY FOR STATIC ARRAY IMPLEMENTATION
bool isFull(Stack * stack);

// FOR DEBUG USE ONLY - REMOVE FROM THE INTERFACE
void show(Stack * stack);
void automaticTest(void);

```

```
#endif //MODULE_H
```

module.c

```
// ADT/stack_array_noencaps/module.c

#include <stdlib.h>
#include <stdio.h>
#include <assert.h>
#include "module.h"

Stack * initStack(){
    Stack * stack= malloc(sizeof(*stack));
    if (!stack) {
        printf("Memory allocation failed!\n");
        exit(EXIT_FAILURE); // Stop the program if malloc fails
    }
    stack->top=-1;
    return stack;
};

void deleteStack(Stack * p_s){
    // attenzione, dangling pointer
    // il chiamante mantiene un puntatore dangling se non lo azzera con p=NULL
    free (p_s);
}

void push(Stack * stack, int val){

    stack->top+=1;
    stack->stackArray[stack->top] = val;
}

// PRECONDIZIONE: stack NON vuoto
int pop(Stack * stack){
    int val = stack->stackArray[stack->top];
    stack->top-=1;
    return val;
}
```

```

// PRECONDIZIONE: stack NON vuoto
int peek(Stack * stack){
    int val = stack->stackArray[stack->top];
    return val;
}

bool isEmpty(Stack * stack){ return stack->top==-1; }

bool isFull(Stack * stack){ return stack->top == STACK_MAX_SIZE-1; }

// FOR DEBUG USE ONLY - REMOVE FROM THE INTERFACE
void show(Stack * stack){
    int i;
    if (isEmpty( stack)) {
        printf("Empty stack!\n");
        return;
    }
    printf("[TOP] ");
    for (i=stack->top; i>=0; i--)
        printf("%d ",stack->stackArray[i]);
    printf("] [BASE]\n");
}

void automaticTest(void) {
    int value;
    Stack * s;
    s = initStack();
    printf("Automatic test\n");
    printf(" - check empty\n");
    assert(isEmpty(s)); //https://en.wikipedia.org/wiki/Assert.h
    printf(" - check push, pop, peek\n");
    // Push 1
    value=1;
    push(s, value);
    assert( !(isEmpty(s)));
    assert(value == peek(s));
    assert( !(isEmpty(s)));
    assert(value == pop(s));
    assert((isEmpty(s)));

    // Push 1, 2, 3
    push(s, 1);

```

```

push(s, 2);
push(s, 3);
printf("->expected: \n[TOP] [3 2 1] [BASE]\n");
printf("->obtained:\n");
show(s);
assert( !(isEmpty(s)) );
assert(3 == peek(s));
assert(3 == pop(s));
assert(2 == peek(s));
assert(2 == pop(s));
assert(1 == peek(s));
assert(1 == pop(s));
assert((isEmpty(s)));

printf("OK!\n");
printf("-----\n");
// Don't forget to free the memory!
deleteStack(s);
}

```

Discussione e miglioramenti

Verifica condizioni di overflow/underflow

Possiamo assegnare alla funzione `pop()` la responsabilità di verificare che lo stack non sia vuoto. `pop()` rende un `bool` per indicare il successo o fallimento dell'operazione, e usa un parametro di output (`int *val`) per rendere il valore.

```
// PRECONDIZIONE: stack NON vuoto
int pop(Stack * stack){
    int val = stack->stackArray[stack->top];
    stack->top-=1;
    return val;
}
```

```
// la responsabilità di verificare che lo stack non sia vuoto è di pop()
bool pop(Stack *stack, int * val){
    if (isEmpty(stack)) {
        return false;
    }

    *val = stack->stackArray[stack->top];
    stack->top--;
    return true;
}
```

Modifiche analoghe vanno applicate a `peek` e `push` per mantenere l'interfaccia coerente.

Stack con capacità dinamica

Abbiamo gestito la dimensione dello stack in modo molto semplice:

```
#define STACK_MAX_SIZE 1000

typedef struct stack {
    int stackArray[STACK_MAX_SIZE];
    int top;
}Stack;
```

Definire la dimensione massima dello stack nell'interfaccia (ad esempio tramite una costante) introduce un limite rigido: tutti gli stack hanno la stessa capacità, fissata a compile-time, e il client non può adattarla alle proprie esigenze. Questo riduce la flessibilità e lega l'uso dell'ADT a una scelta implementativa.

Una soluzione è permettere all'utente di specificare la capacità al momento della creazione dello stack. In questo modo, la funzione `initStack` deve essere modificata per allocare **dinamicamente** anche il vettore che contiene gli elementi, che prima era statico. L'array non è più contenuto direttamente nella struttura, ma viene allocato separatamente in memoria dinamica. Si introduce quindi una **doppia allocazione dinamica**, una per la struttura `Stack` e una per il vettore di dati.

Di conseguenza, anche `deleteStack` deve essere aggiornata per liberare correttamente entrambe le aree di memoria.

- **Struttura dati**

```
typedef struct stack {  
    int *data;  
    int top;  
    int capacity;  
} Stack;
```

- **Creazione e distruzione**

```
#include <stdlib.h>  
  
Stack *initStack(int capacity) {  
    if (capacity <= 0) return NULL;  
  
    Stack *s = malloc(sizeof(Stack));  
    if (s == NULL) return NULL;  
  
    s->data = malloc(capacity * sizeof(int));  
    if (s->data == NULL) {  
        free(s);  
        return NULL;  
    }  
  
    s->top = -1;  
    s->capacity = capacity;  
  
    return s;  
}  
  
void deleteStack(Stack *s) {  
    if (s == NULL) return;  
  
    free(s->data);
```

```
    free(s);  
}
```

- **Conseguenze sul resto del codice**

Poiché la capacità non è più una costante ma un campo della struttura, anche le funzioni che dipendono da essa devono essere aggiornate. In particolare, `isFull` deve confrontare `top` con `capacity - 1`, invece di usare una dimensione fissata a compile-time.

Rendere dinamica la capacità dello stack aumenta la flessibilità, ma introduce una gestione della memoria più complessa e richiede maggiore attenzione nell'implementazione.

- **Esempio d'uso**

```
Stack *s = initStack(50);
```

Dangling pointer

Una funzione come

```
void deleteStack(Stack *s) {  
    free(s);  
}
```

libera correttamente la memoria puntata da `s`, ma **non modifica il puntatore del chiamante**. Di conseguenza, dopo la chiamata, nel `main` la variabile `s` continua a contenere l'indirizzo di una zona di memoria ormai liberata: si ha quindi un **dangling pointer**. Un puntatore dangling è pericoloso perché può essere usato accidentalmente (dal `main`) dopo la `free`, producendo comportamento indefinito.

Per evitare il problema, una prima soluzione è lasciare la responsabilità al chiamante, che deve mettere a `NULL` il puntatore.

```
// main  
  
deleteStack(s);  
s=NULL;
```

Una seconda soluzione, più robusta, è passare alla funzione **l'indirizzo del puntatore**, in modo che la funzione possa azzerarlo direttamente:

```
void deleteStack(Stack **s) {
    if (s == NULL || *s == NULL) return;
    free(*s);
    *s = NULL;
}
```

La chiamata in C è:

```
deleteStack(&s);
```

Se lo stack contiene anche memoria dinamica interna, per esempio un array **data**, allora prima di **free(*s)** bisogna liberare anche quella memoria.

Stack con Array Statico, incapsulamento completo

Nel progetto precedente abbiamo già adottato diverse scelte che rendono naturale il passaggio all'incapsulamento completo: il **main** manipola solo puntatori a **Stack**, la creazione e la distruzione della struttura avvengono tramite funzioni del modulo. Per questo motivo, il passaggio a un ADT con incapsulamento completo è quasi immediato: è sufficiente spostare la definizione completa della struttura nel file **.c** e lasciare nel **.h** solo la dichiarazione del tipo incompleto e delle funzioni dell'interfaccia.

In questo modo, la rappresentazione interna dello stack non è più accessibile al codice client, che può usare la struttura solo attraverso le operazioni fornite dal modulo. La sola vera condizione è che il codice client non acceda più direttamente ai campi della struttura.

Applichiamo anche le principali migliorie discusse, come la capacità dinamica, l'aggiornamento di **deleteStack** e i controlli sulle allocazioni.

Rimane il problema del dangling pointer, di cui abbiamo già discusso. Il codice finale è disponibile sul repository ed è riportato anche di seguito.

module.h

```
// ADT/stack_array_encaps/module.h

/* STACK
 * Implementazione: array dinamico
 * CON incapsulamento completo
 *
 */

#ifndef MODULE_H
```

```

#define MODULE_H
#include <stdbool.h>

typedef struct stack Stack;

// Function prototypes

// Push element to the top of the stack

bool push(Stack * stack, int val);

// Remove the top most element of the stack and store it in val
bool pop(Stack * stack, int * val);

// Return the top most element of the stack and store it in val
bool peek(Stack * stack, int * val);

// Check if the stack is in Underflow state or not
bool isEmpty(Stack * stack);

Stack * initStack(int capacity); //INIT
void deleteStack(Stack * stack); // Free memory

// Check if the stack is in Overflow state or not
bool isFull(Stack * stack);

// FOR DEBUG USE ONLY - REMOVE FROM THE INTERFACE
void show(Stack * stack);
void automaticTest(void);

#endif //MODULE_H

```

module.c

```

// ADT/stack_array_encaps/module.c

#include <stdlib.h>
#include <stdio.h>
#include <assert.h>

```

```

#include "module.h"

struct stack {
    int *data;
    int top;
    int capacity;
};

Stack *initStack(int capacity) {
    if (capacity <= 0) return NULL;

    Stack *s = malloc(sizeof(Stack));
    if (s == NULL) return NULL;

    s->data = malloc(capacity * sizeof(int));
    if (s->data == NULL) {
        free(s);
        return NULL;
    }

    s->top = -1;
    s->capacity = capacity;

    return s;
}

void deleteStack(Stack *s) {
    if (s == NULL) return;

    free(s->data);
    free(s);
}

bool push(Stack * stack, int val){
    if (isFull(stack)) {
        return false;
    }

    stack->top+=1;
    stack->data[stack->top] = val;
    return true;
}

bool pop(Stack *stack, int * val){

```

```

    if (val == NULL || isEmpty(stack)) {
        return false;
    }

    *val = stack->data[stack->top];
    stack->top--;
    return true;
}

bool peek(Stack * stack, int * val){
    if (val == NULL || isEmpty(stack)) {
        return false;
    }

    *val = stack->data[stack->top];
    return true;
}

bool isEmpty(Stack * stack){ return stack->top== -1; }

bool isFull(Stack * stack){ return stack->top == stack->capacity-1; }

// FOR DEBUG USE ONLY - REMOVE FROM THE INTERFACE
void show(Stack * stack){
    int i;
    if (isEmpty( stack)) {
        printf("Empty stack!\n");
        return;
    }
    printf("[TOP] ");
    for (i=stack->top; i>=0; i--)
        printf("%d ",stack->data[i]);
    printf("] [BASE]\n");
}

void automaticTest(void) {
    int value;
    int peekedValue;
    int poppedValue;
    Stack * s;
    s = initStack(10);
    assert(s != NULL);

```

```

printf("Automatic test\n");
printf(" - check empty\n");
assert(isEmpty(s)); //https://en.wikipedia.org/wiki/Assert.h
printf(" - check push, pop, peek\n");
// Push 1
value=1;
assert(push(s, value));
assert( !isEmpty(s));
assert(peek(s, &peekedValue));
assert(value == peekedValue);
assert( !isEmpty(s));
assert(pop(s, &poppedValue));
assert(value == poppedValue);
assert((isEmpty(s)));

// Push 1, 2, 3
assert(push(s, 1));
assert(push(s, 2));
assert(push(s, 3));
printf("->expected: \n[TOP] [3 2 1] [BASE]\n");
printf("->obtained:\n");
show(s);
assert( !isEmpty(s));
assert(peek(s, &peekedValue));
assert(3 == peekedValue);
assert(pop(s, &poppedValue));
assert(3 == poppedValue);
assert(peek(s, &peekedValue));
assert(2 == peekedValue);
assert(pop(s, &poppedValue));
assert(2 == poppedValue);
assert(peek(s, &peekedValue));
assert(1 == peekedValue);
assert(pop(s, &poppedValue));
assert(1 == poppedValue);
assert((isEmpty(s)));

printf("OK!\n");
printf("-----\n");
// Don't forget to free the memory!
deleteStack(s);
}

```

💡 Esercizi

→ **Esercizio.** Implementate ‘da zero’ lo stack basato su array con incapsulamento completo

→ **Esercizio.** Calcolo della media nello stack

Aggiungere allo stack una funzione che calcoli la **media degli elementi contenuti**.

Esistono diverse possibili strategie di implementazione, con diversi compromessi tra semplicità ed efficienza:

- **Compute on demand** La media viene calcolata ogni volta che la funzione è chiamata, iterando su tutti gli elementi dello stack.
 - implementazione semplice
 - costo lineare ($O(n)$) ad ogni chiamata
- **Running mean (media incrementale).** La media (o la somma) viene aggiornata ogni volta che si esegue una **push** o una **pop**. Il valore è memorizzato nella struttura.
 - accesso immediato alla media ($O(1)$)
 - maggiore complessità: bisogna aggiornare correttamente il valore ad ogni operazione
- **Lazy evaluation.** La media viene calcolata solo alla prima richiesta e poi memorizzata. Quando lo stack cambia, la media viene invalidata e ricalcolata alla successiva richiesta.
 - evita calcoli inutili
 - efficiente se la media è richiesta raramente o mai, o se richiesta spesso ma senza aggiornamenti dello stack.
 - richiede la gestione di uno stato “valido/non valido” (es. flag)

Variante: minimo, massimo... ***

→ **Esercizio.** Modificare l’implementazione dello stack per supportare l’operazione ‘reverse()’

(COMPLESSO)

Aggiungere un’operazione **reverse()** che inverta l’ordine degli elementi attualmente presenti nello stack. Dopo l’inversione, lo stack deve continuare a funzionare correttamente con le operazioni standard (**push**, **pop**, ecc.).

L’inversione deve essere effettuata **in-place**, cioè **senza utilizzare strutture dati ausiliarie** (come altri stack o array), ma solo un numero limitato di variabili temporanee.

Possibili soluzioni:

- **Soluzione 1:** Utilizzare la ricorsione per rimuovere gli elementi e reinserirli in ordine inverso (eventualmente con una funzione ausiliaria come `pushAtBottom`).
Nota: questa soluzione è semplice ma utilizza implicitamente la call stack.

Traccia

```
void pushAtBottom(Stack* stack, int value) {
    if (isEmpty(stack)) {
        push(stack, value);
        return;
    }

    int temp = pop(stack);
    pushAtBottom(stack, value);
    push(stack, temp);
}

void reverse(Stack* stack) {
    if (isEmpty(stack)) {
        return;
    }

    int temp = pop(stack);
    reverse(stack);
    pushAtBottom(stack, temp);
}
```

- **Soluzione 2:** Modificare l'implementazione dello stack per supportare l'operazione `reverse()`

Nell'implementazione standard su array, la base dello stack è sempre indicata (implicitamente) dall'indice 0. Possiamo introdurre un nuovo indice **base** che denota esplicitamente la base dello stack. Se invertiamo il ruolo di **base** e **top**, e adattiamo di conseguenza il funzionamento di `push`, `pop`, `peek`, lo stack “appare” invertito, ma **senza spostare fisicamente gli elementi**: l'operazione può quindi essere molto efficiente.

Per ottenere questo comportamento dobbiamo reinterpretare lo stack come una struttura su array con gestione **circolare** degli indici.

In particolare:

- gli indici devono essere aggiornati in modo **modulare** (quando si supera `capacity - 1`, si torna a 0 e viceversa)
- le condizioni di stack vuoto e pieno non possono più essere espresse con semplici confronti lineari, ma devono tener conto della relazione tra **base** e **top**

- può essere necessario introdurre una nozione di **direzione logica** (esplicita o implicita) per interpretare correttamente l'ordine degli elementi

Di fatto, questa soluzione introduce una logica simile a quella di un **buffer circolare**.

La seconda soluzione è più efficiente (l'inversione è “logica”, non fisica), ma introduce una complessità maggiore e si allontana dall'implementazione classica dello stack. Non è la soluzione ‘ideale’ per questo esercizio.

Implementazione dello Stack con Lista Concatenata

Uno stack può essere implementato mediante una **lista concatenata (linked list)**, in cui ogni elemento (**nodo**) contiene un valore e un puntatore al nodo successivo. La cima dello stack è individuata da un puntatore (**head** o **top**) al primo nodo della lista. Le operazioni di inserimento e rimozione aggiornano questo puntatore.

Componenti principali

- **Nodo (Node)**

Un nodo è una struttura che contiene:

- **value**: il valore memorizzato nello stack
- **next**: un puntatore al nodo successivo nella lista

- **Puntatore alla cima (head o top)**

È un puntatore che indica il nodo in cima allo stack.

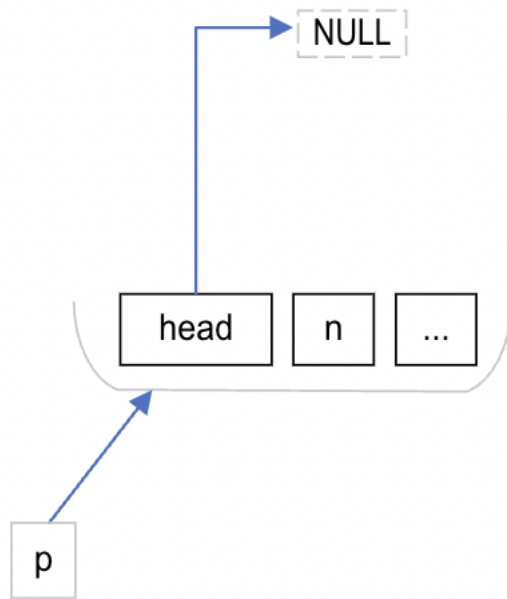
- quando lo stack è vuoto → **head == NULL**
- quando si inserisce o rimuove un elemento → **head** viene aggiornato

In C, è buona pratica definire una struttura per il **nodo** ed una struttura per lo **stack**, che contiene almeno il puntatore alla cima. Opzionalmente la struttura dello stack può contenere un campo (nell'esempio, **numberOfElements**) utile per conoscere la dimensione dello stack in tempo costante. .

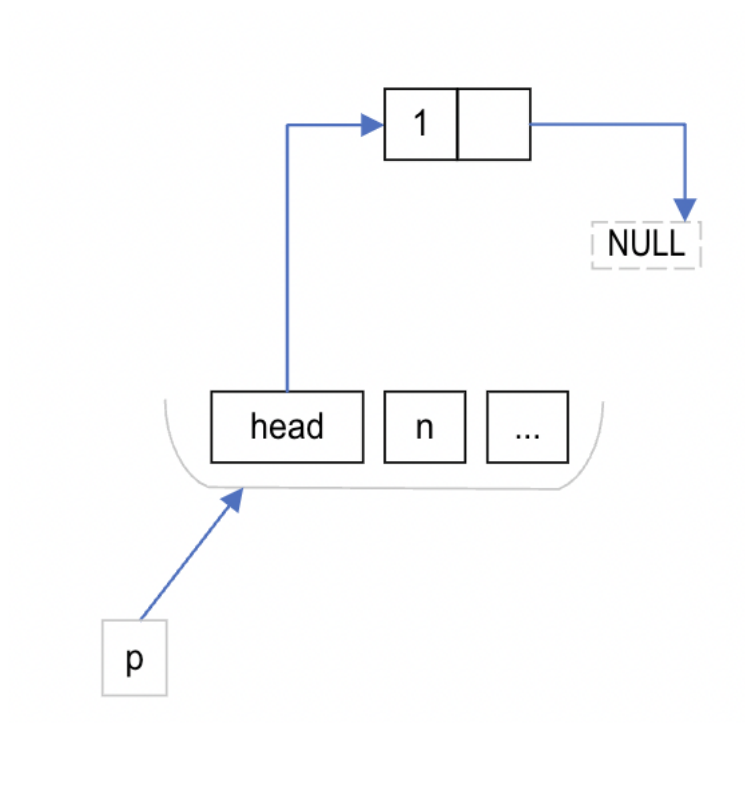
```
typedef struct node {  
    int value;  
    struct node *next;  
} Node;  
  
typedef struct stack {  
    Node *head;  
    int numberOfElements; // opzionale  
    // altri campi opzionali  
} Stack;
```

A differenza dell'implementazione con array, lo stack basato su lista concatenata non ha una dimensione massima fissa: il limite è dato solo dalla memoria disponibile.

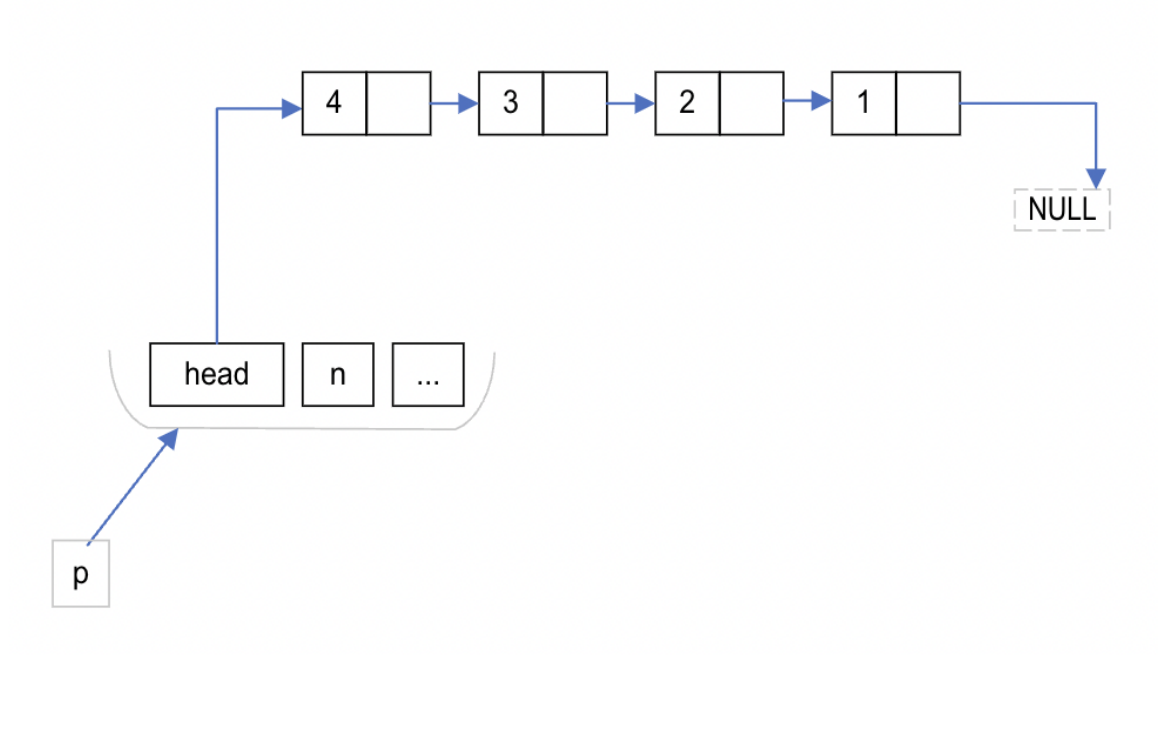
Stack vuoto



Stack con un singolo elemento



Stack con vari elementi



Interfaccia

Nel caso dell'implementazione dello stack mediante **lista concatenata**, l'interfaccia può essere definita come segue:

```
Stack *initStack(void);

bool push(Stack *stack, int val);
bool pop(Stack *stack, int *val);
bool peek(const Stack *stack, int *val);

bool isEmpty(const Stack *stack);

void deleteStack(Stack *stack);
```

Questa interfaccia è simile a quella già vista per lo stack implementato con array, ma con alcune differenze importanti.

- `initStack()` non deve fissare una dimensione massima dello stack, perché nell'implementazione con lista concatenata lo stack non ha una capacità predefinita: i nodi vengono allocati dinamicamente man mano che servono.
- `push()` non può fallire per **stack pieno**, come accadeva nell'implementazione con array statico. Può fallire solo se l'allocazione dinamica di un nuovo nodo (`malloc`) non va a buon fine.
- per lo stesso motivo, in questa implementazione la funzione `isFull()` non viene normalmente introdotta: non esiste infatti un limite massimo intrinseco della struttura. L'unico limite reale è la memoria disponibile, ma questo non corrisponde a una nozione di "pieno" gestibile in modo semplice e significativo tramite una funzione come `isFull()`.
- `deleteStack()` deve liberare non solo la struttura `Stack`, ma anche tutti i nodi della lista.

Nota su const

Nelle funzioni come `peek(const Stack *stack, int *val)` e `isEmpty(const Stack *stack)`, la parola chiave `const` indica che la funzione **non deve modificare** l'oggetto puntato da `stack`.

Questo non è una particolarità di questa implementazione dello stack: è una buona pratica generale in C. Quando si passa un puntatore a un oggetto **solo per efficienza**, ma non si vuole permettere alla funzione di modificarlo, è opportuno usare `const`. In questo modo l'interfaccia è più chiara e il compilatore può aiutare a prevenire modifiche accidentali.

Esempio di main

(Nel repository github il main completo)

```
#include "module.h"
#include <stdio.h>

int main(void) {
    Stack *s = initStack();
    int value;

    if (s == NULL) {
        printf("Errore di allocazione dello stack.\n");
        return 1;
    }

    push(s, 10);
    push(s, 20);
    push(s, 30);

    if (peek(s, &value)) {
        printf("Top: %d\n", value);
    }

    while (pop(s, &value)) {
        printf("Pop: %d\n", value);
    }

    deleteStack(s);
    return 0;
}
```

Implementazione delle funzioni fondamentali

- **Controllo delle precondizioni**

Nelle implementazioni robuste, le funzioni che operano sullo stack devono verificare alcune precondizioni, in particolare:

- che il puntatore allo stack sia valido (`stack != NULL`)
- che eventuali parametri di output siano validi (`val != NULL`)
- che lo stack non sia vuoto (quando richiesto)

Ad esempio, una funzione come pop o peek può effettuare un controllo del tipo:

```
if (stack == NULL || val == NULL || isEmpty(stack)) {
    return false;
}
```

In queste dispense, per semplicità e per focalizzarsi sugli aspetti fondamentali dell'algoritmo, tali controlli possono essere omessi nelle prime versioni del codice. Tuttavia, è importante ricordare che in un'implementazione reale questi controlli sono essenziali per evitare comportamenti indefiniti.

initStack

La funzione `initStack` alloca dinamicamente una nuova struttura `Stack` e la inizializza in uno stato consistente: lo stack è vuoto (`head = NULL`) e il numero di elementi è posto a zero. Restituisce un puntatore allo stack appena creato, che dovrà essere successivamente liberato con `deleteStack`.

```
Stack *initStack(void) {
    Stack *stack = malloc(sizeof(Stack));
    stack->head = NULL;
    stack->numberOfElements = 0;

    return stack;
}
```

isEmpty

La funzione `isEmpty` verifica se lo stack è vuoto controllando se il puntatore alla testa (`head`) è `NULL`. Per maggiore robustezza, considera vuoto anche il caso in cui il puntatore allo stack sia `NULL`.

```
bool isEmpty(const Stack *stack) {
    return stack == NULL || stack->head == NULL;
}
```

push

La funzione `push` inserisce un nuovo elemento **in cima allo stack**. Nell'implementazione con lista concatenata, questo significa creare un nuovo nodo e inserirlo **all'inizio della lista**. Il meccanismo è lo stesso sia in caso di stack inizialmente vuoto sia in caso di stack che contiene già elementi.

- viene allocato dinamicamente un nuovo nodo
- se l'allocazione fallisce, la funzione restituisce **false**.
- se ha successo, il nuovo nodo viene inizializzato assegnando **val** al campo **value**.
- Successivamente, il campo **next** del nuovo nodo viene fatto puntare all'attuale testa della lista (**stack->head**): in questo modo il nuovo nodo si collega al vecchio contenuto dello stack. Infine, **stack->head** viene aggiornato in modo da puntare al nuovo nodo, che diventa quindi la nuova cima dello stack, e il contatore degli elementi viene incrementato.

Il punto chiave è che l'inserimento avviene sempre in testa alla lista, quindi **push** ha costo **O(1)**.

```
bool push(Stack *stack, int val) {

    // 1 *****
    Node *newNode = malloc(sizeof(Node));
    if (newNode == NULL) {
        return false;
    }
    // 2 *****
    newNode->value = val;
    newNode->next = stack->head;

    // 3 *****
    stack->head = newNode;
    stack->numberOfElements++;

    return true;
}
```

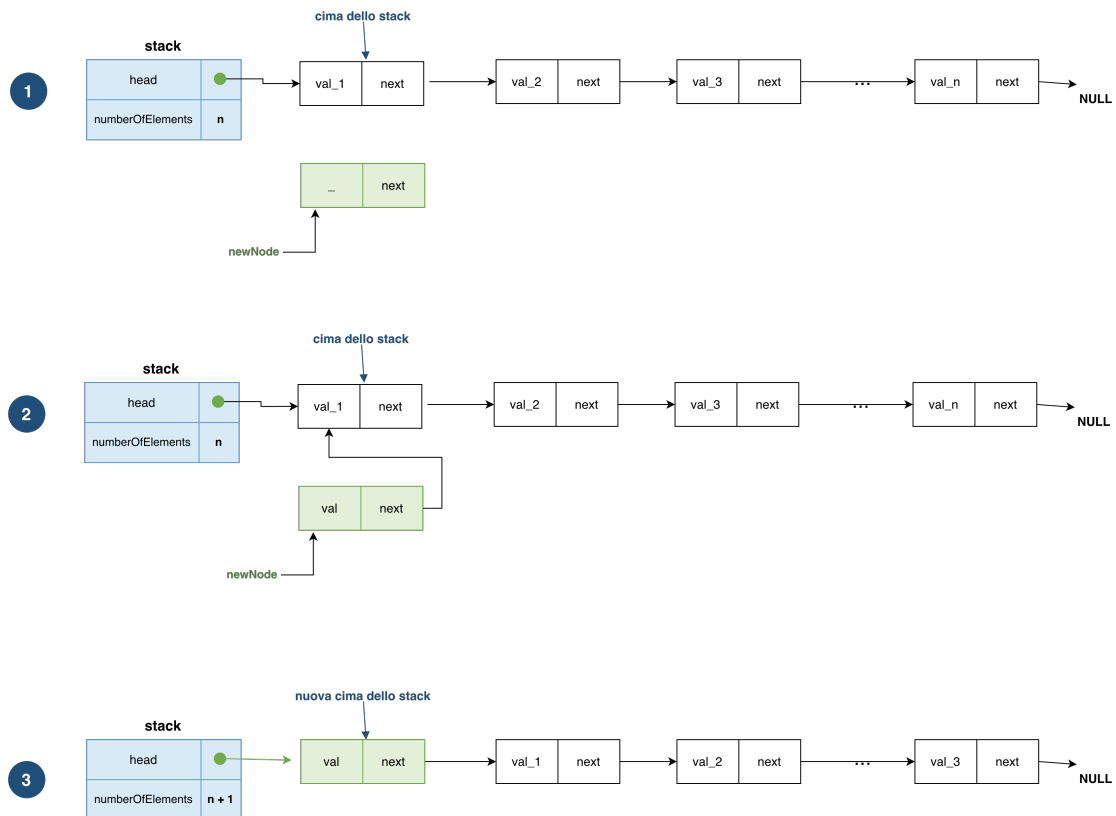


Figure 1: push_nonempty.png

pop

La funzione `pop` rimuove l'elemento in cima allo stack. Nell'implementazione con lista concatenata, questo significa salvare il valore del nodo puntato da `head`, aggiornare `head` al nodo successivo e liberare la memoria del nodo rimosso.

Anche `pop`, come `push`, opera sempre sulla testa della lista: per questo motivo ha costo $O(1)$.

```
bool pop(Stack *stack, int *val) {

    // 1 *****
    Node *temp = stack->head;
    *val = temp->value;

    // 2 *****
    stack->head = temp->next;

    // 3 *****
}
```

```

stack->numberOfElements--;
free(temp);
return true;
}

```

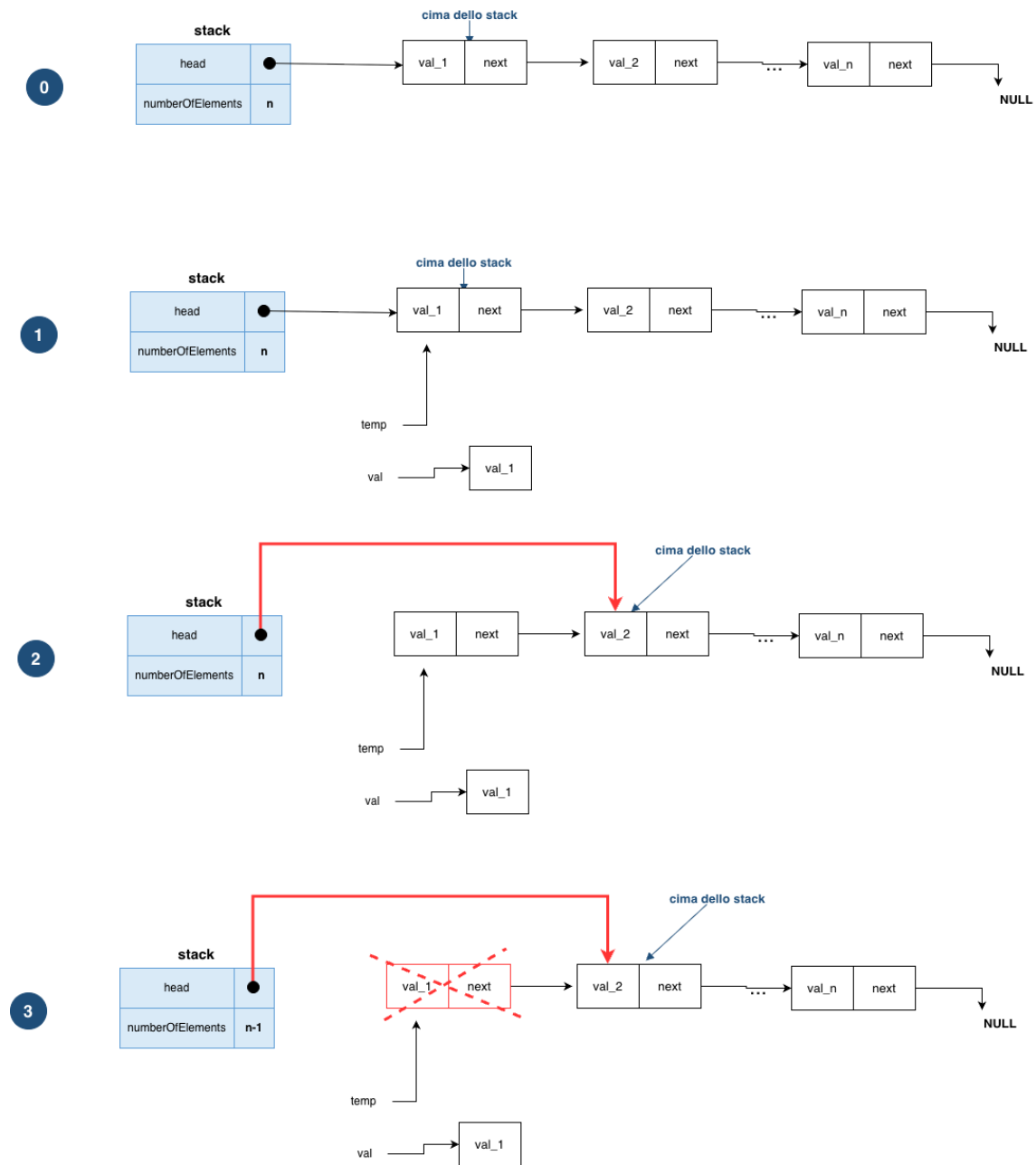


Figure 2: pop_nonempty.png

peek

La funzione `peek` legge il valore dell'elemento in cima allo stack senza rimuoverlo. Accede al nodo puntato da `head` e copia il valore nel parametro di output.

```
bool peek(const Stack *stack, int *val) {  
  
    *val = stack->head->value;  
    return true;  
}
```

deleteStack

La funzione `deleteStack` libera tutta la memoria associata allo stack: attraversa la lista nodo per nodo, deallocando ciascun elemento, e infine libera la struttura `Stack`.

```
void deleteStack(Stack *stack) {  
  
    // 1 *****  
    Node *current = stack->head;  
  
    while (current != NULL) {  
  
        // 2 *****  
        Node *next = current->next;  
  
        // 3 *****  
        free(current);  
        current = next;  
    }  
    // 4 *****  
    free(stack);  
}
```

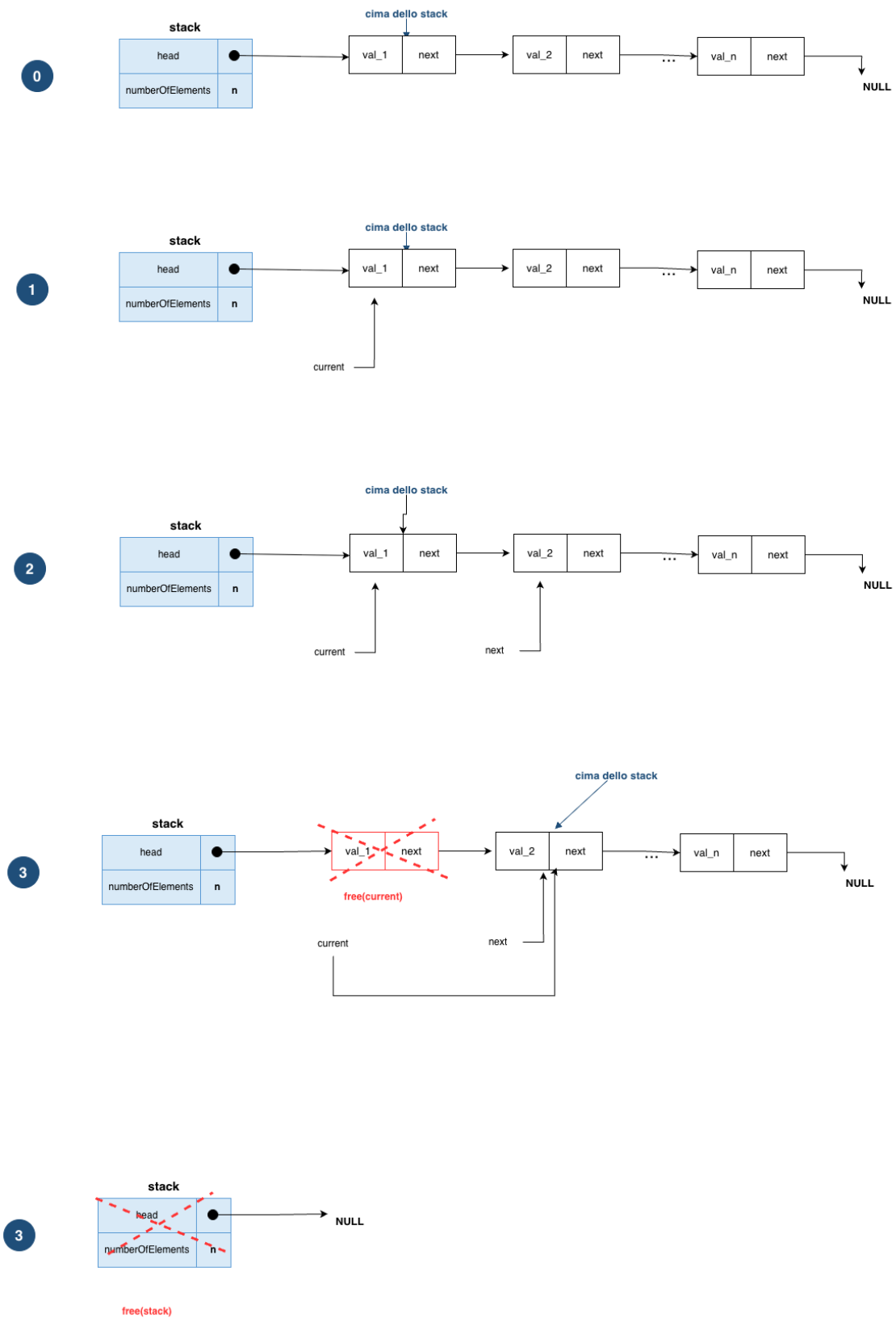


Figure 3: delete_nonempty.png

Il codice completo è disponibile nel repository GitHub, nella directory `src/ADT/stack_linked_list_noencaps`

Stack con Lista Concatenata, incapsulamento completo

Esattamente come per lo stack basato su array, dato il modo in cui abbiamo implementato il codice, è possibile ottenere un incapsulamento completo spostando la definizione della struttura nel file `.c` e lasciando nel `.h` solo la dichiarazione del tipo incompleto e delle funzioni dell'interfaccia.

Il codice completo è disponibile nel repository GitHub, nella directory `src/ADT/stack_linked_list_encaps`

💡 Ulteriori esercizi sullo stack

Gli esercizi proposti per lo stack basato su array possono essere risolti anche utilizzando uno stack implementato con lista concatenata.

Di seguito se ne propongono altri.

→ **Esercizio.** Aggiungere l'operazione `reverse()` allo stack, eventualmente modificando l'implementazione.

Si richiede di implementare un'operazione `reverse()` che inverta l'ordine degli elementi presenti nello stack. Dopo l'inversione, lo stack deve continuare a funzionare correttamente con le operazioni standard (`push`, `pop`, ecc.).

L'inversione deve essere effettuata **in-place**, cioè **senza utilizzare strutture dati ausiliarie** (come altri stack o array), ma solo un numero limitato di variabili temporanee. La soluzione proposta differisce da quella per lo stack implementato mediante array.

Soluzione

Nel caso di uno stack implementato tramite **lista concatenata**, l'inversione può essere realizzata invertendo i puntatori tra i nodi della lista.

Si percorre la lista e, per ogni nodo, si modifica il campo `next` in modo che punti al nodo precedente invece che al successivo. Al termine del processo, il puntatore `head` viene aggiornato all'ultimo nodo visitato, che diventa la nuova cima dello stack.

Questa operazione non richiede memoria aggiuntiva significativa (solo pochi puntatori temporanei) e ha costo **O(n)**

```
void reverse(Stack *stack) {
    if (stack == NULL || stack->head == NULL) {
        return;
    }

    Node *prev = NULL;
    Node *current = stack->head;
    Node *next = NULL;

    while (current != NULL) {
        next = current->next;    // salva il prossimo nodo
        current->next = prev;    // inverte il collegamento
        prev = current;          // avanza prev
        current = next;          // avanza current
    }
}
```

```
    stack->head = prev; // aggiorna la nuova cima
}
```

L'operazione modifica solo i collegamenti tra i nodi, senza spostare fisicamente i dati. La soluzione è generale e funziona per qualsiasi lista concatenata semplicemente collegata.

Da notare che questa soluzione - come le precedenti - **bypassa l'interfaccia dello stack**.

Per usare l'interfaccia dello stack posso usare la ricorsione

```
// Inserisce un elemento sul fondo dello stack
void insertAtBottom(Stack *stack, int value) {
    int top;

    if (isEmpty(stack)) {
        push(stack, value);
        return;
    }

    pop(stack, &top);
    insertAtBottom(stack, value);
    push(stack, top);
}

// Inverte lo stack usando solo push/pop
void reverse(Stack *stack) {
    int top;

    if (isEmpty(stack)) {
        return;
    }

    pop(stack, &top);
    reverse(stack);
    insertAtBottom(stack, top);
}
```

- **reverse** rimuove un elemento alla volta dalla cima, si richiama ricorsivamente, poi reinserisce l'elemento **sul fondo**.
- **insertAtBottom**, non potendo accedere direttamente al fondo, svuota temporaneamente lo stack, inserisce il valore, e rimette tutto.

Complessità:

- Tempo $O(n^2)$ — per ogni elemento, `insertAtBottom` percorre tutto lo stack
- Spazio $O(n)$ — call stack della ricorsione

L'approccio è più costoso, ma rispetta l'astrazione dello stack usando solo `push`, `pop` e `isEmpty`, senza toccare la linked list internamente.

Applicazioni dello stack - esercizi

→ Esercizio. Verifica di parentesi bilanciate usando uno stack

Scrivere una funzione `check_balanced_parentheses(expr)` che verifichi se le parentesi presenti in una espressione `expr` sono bilanciate usando uno stack. Il programma deve leggere un'espressione contenente parentesi tonde `()`, quadre `[]` e graffe `{}` e analizzarla carattere per carattere.

- Se il carattere è una parentesi aperta, cioè `(`, `[` o `{`, viene inserito nello stack.
- Se il carattere è una parentesi chiusa, cioè `)`, `]` o `}`, bisogna verificare:
 - che lo stack non sia vuoto;
 - che l'elemento in cima allo stack sia la corrispondente parentesi aperta.
- Se una di queste condizioni non è soddisfatta, l'espressione non è bilanciata.

Tutti i caratteri diversi da parentesi sono ignorati.

Al termine della scansione:

- se lo stack è vuoto, l'espressione è bilanciata;
- altrimenti, l'espressione non è bilanciata.

Casi particolari

- Un'espressione vuota deve essere considerata bilanciata.
- Il programma deve gestire correttamente parentesi non corrispondenti o non chiuse.
- `main` per testare la funzione

```
#include <stdio.h>

int main() {
    const char *expr1 = "{[()]}" ;
    const char *expr2 = "{[( )]}" ;

    if (check_balanced_parentheses(expr1)) {
        printf("The expression '%s' is balanced.\n", expr1);
    }
}
```

```

    } else {
        printf("The expression '%s' is not balanced.\n", expr1);
    }

    if (check_balanced_parentheses(expr2)) {
        printf("The expression '%s' is balanced.\n", expr2);
    } else {
        printf("The expression '%s' is not balanced.\n", expr2);
    }

    return 0;
}

```

- Possibile soluzione

```

bool is_open(char c) {
    return c == '(' || c == '[' || c == '{';
}

bool is_close(char c) {
    return c == ')' || c == ']' || c == '}';
}

bool match_parentheses(char open, char close) {
    return (open == '(' && close == ')') ||
           (open == '[' && close == ']') ||
           (open == '{' && close == '}');
}

bool check_balanced_parentheses(const char *expression) {
    Stack *stack = initStack();

    for (int i = 0; expression[i] != '\0'; i++) {
        char current = expression[i];

        if (is_open (current)) {
            // se push fallisce, termina con false
            if (!push(stack, current)) {
                deleteStack(stack);
                return false;
            }
        }
        else if (is_close (current)) {
            int top;
            // se pop fallisce, termina con false

```

```

        if (!pop(stack, &top)) {
            deleteStack(stack);
            return false;
        }

        if (!match_parentheses (top, current)){
            deleteStack(stack);
            return false;
        }
        // tutti i caratteri diversi da parentesi sono ignorati
    }
}

// al termine della scansione lo stack deve essere vuoto

bool balanced = isEmpty(stack);
deleteStack(stack);

return balanced;
}

```

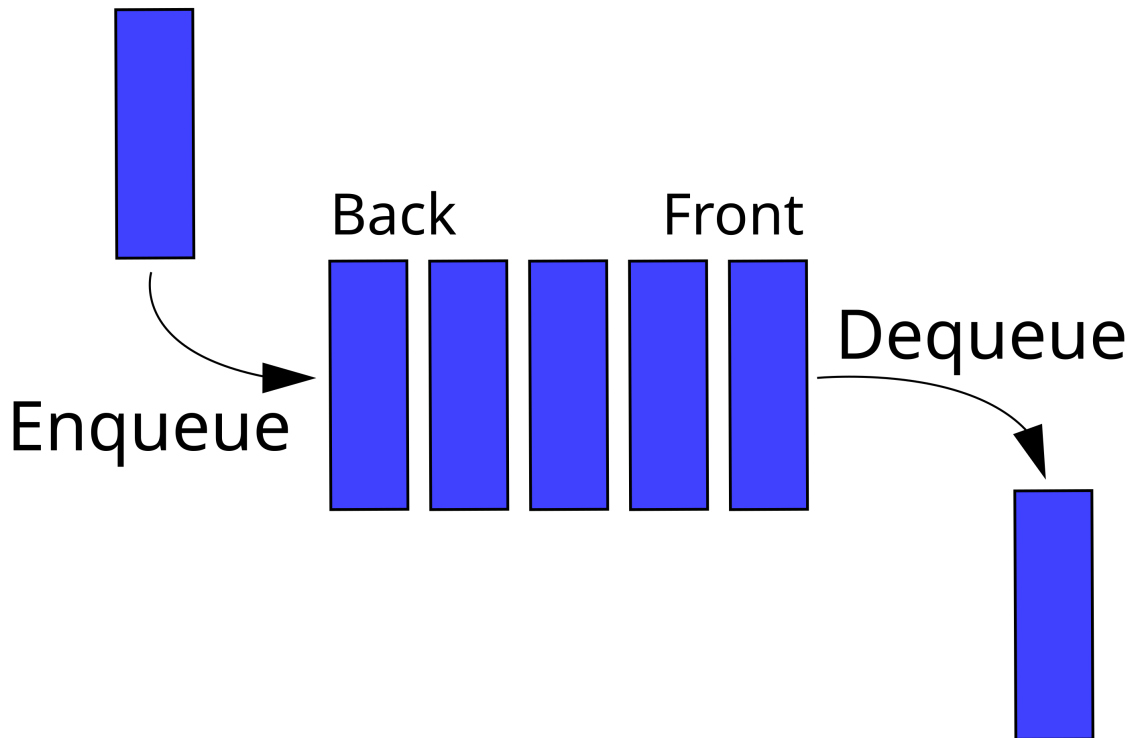
→ **Esercizio.** Invertire una stringa usando uno stack

QUEUE

Politica FIFO: *First In, First Out*

[https://en.wikipedia.org/wiki/Queue_\(abstract_data_type\)](https://en.wikipedia.org/wiki/Queue_(abstract_data_type))

Il codice utilizzato è disponibile nel repository alla cartella `src/ADT/queue_*`



Una **queue** (coda) è un tipo di dato astratto (ADT) lineare in cui gli elementi sono gestiti secondo la politica **FIFO** (*First In, First Out*): il primo elemento inserito è anche il primo a essere rimosso.

Un'analogia intuitiva è quella di una **fila** (ad esempio allo sportello): chi arriva per primo viene servito per primo.

A differenza dello **stack** (LIFO), in cui tutte le operazioni avvengono sulla stessa estremità, nella queue le operazioni principali agiscono su **due estremità distinte**:

- **front**: da cui si rimuovono gli elementi
- **tail (o rear)**: in cui si inseriscono nuovi elementi

Questo modello rende la queue particolarmente adatta a scenari in cui è necessario **preservare l'ordine temporale** degli elementi (buffer, code di processi, gestione eventi, ecc.).

Operazioni fondamentali

Le operazioni che definiscono il comportamento della queue sono:

- **init()**
Crea e inizializza una nuova coda vuota

- `isEmpty()`
Verifica se la coda è vuota
 - `enqueue()` (o `put()`)
Inserisce un nuovo elemento **in fondo** alla coda
 - `dequeue()` (o `get()`)
Rimuove e restituisce l'elemento **in testa** alla coda
 - `peek()` (o `front()`)
Restituisce l'elemento in testa senza rimuoverlo
-

Osservazioni

- Le operazioni **enqueue** e **dequeue** devono essere efficienti (tipicamente $O(1)$).
- Come per lo stack, l'ADT queue è definito dal **comportamento FIFO**, indipendentemente dalla sua implementazione concreta.
- In C, la queue può essere implementata:
 - mediante **array**
 - mediante **lista concatenata**

Le due implementazioni differiscono per gestione della memoria, flessibilità e complessità, ma condividono la stessa interfaccia astratta.

Implementazione della Queue mediante array

Una queue può essere implementata utilizzando un **array**, analogamente allo stack. Tuttavia, a differenza dello stack, la queue richiede la gestione di **due estremità**:

- **FRONT**: indice dell'elemento in testa (da cui si estrae)
- **TAIL**: indice dell'ultimo elemento inserito (in cui si inserisce)

Una prima implementazione “lineare” della queue utilizza un array in cui **FRONT** e **TAIL** avanzano solo verso destra. Tuttavia, dopo alcune operazioni di **dequeue**, gli elementi in testa vengono rimossi e lo spazio nelle prime posizioni dell'array rimane inutilizzato.

Ad esempio, con un array di dimensione 5:

[10, 20, 30, 40, 50]
↑ ↑
FRONT = 0, TAIL = 4

Dopo tre dequeue:

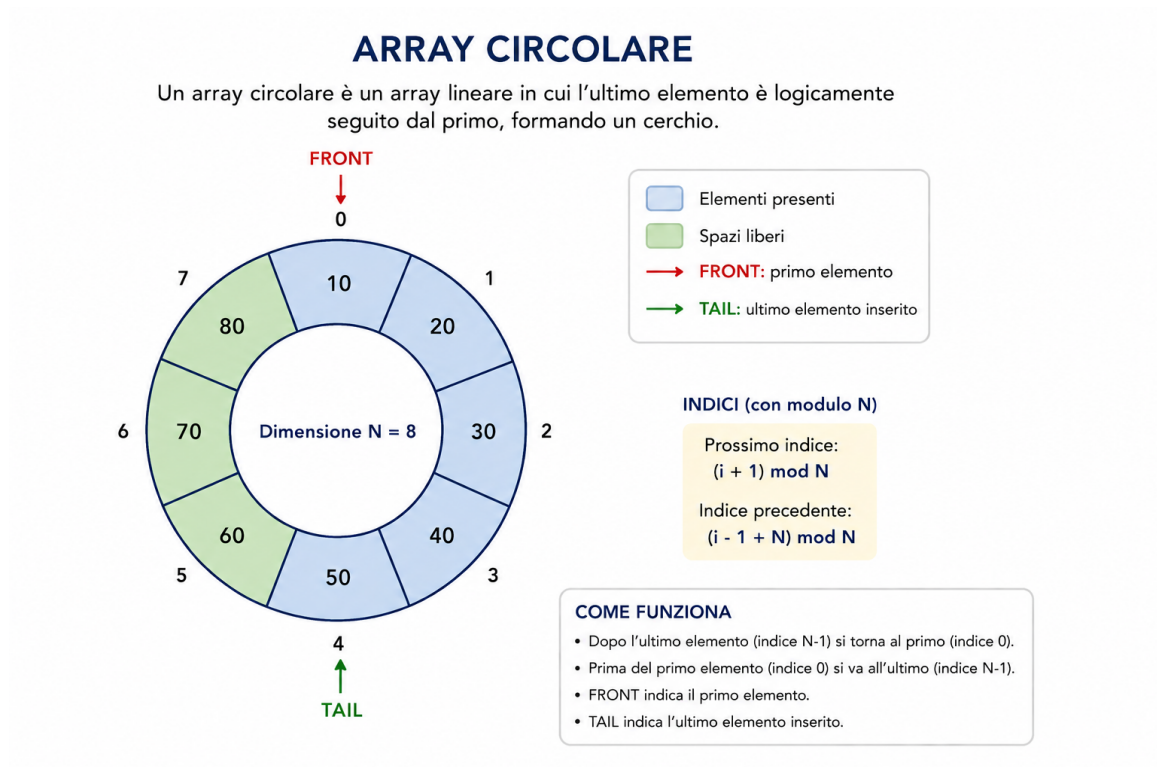
[_ , _ , _ , 40, 50]
 ↑ ↑
FRONT = 3, TAIL = 4

Le prime tre celle sono libere, ma non possono essere riutilizzate. Se si esegue un **enqueue**, **TAIL** dovrebbe avanzare oltre l'array, causando un overflow anche se c'è spazio disponibile. Questo problema è chiamato **falso overflow**: l'array non è realmente pieno, ma l'implementazione lineare non riesce a riutilizzare lo spazio liberato.

Per risolvere questo problema si utilizza un **array circolare (circular buffer)**.

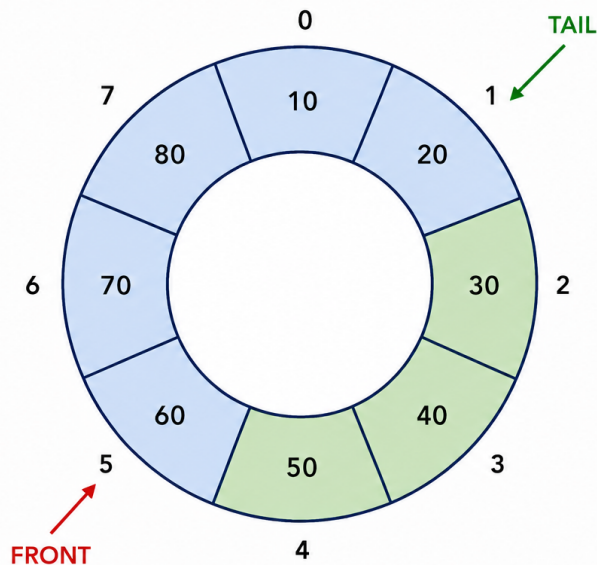
Array circolare

L'idea è considerare l'array come se fosse "chiuso su sé stesso": quando si raggiunge l'ultima posizione, si ricomincia da zero.



10	20	30	40	50	60	70	80
0	1	2	3	4	5	6	7

FRONT = 0 TAIL = 4



Stato dell'array circolare dopo varie operazioni di inserimento (enqueue) e rimozione (dequeue).

10	20	30	40	50	60	70	80
0	1	2	3	4	5	6	7

TAIL = 1 FRONT = 5

Questo comportamento si ottiene usando l'operatore **modulo**, ed incrementando l'indice in questo modo

```
index = (index + 1) % capacity
```

Esempio:

```

capacity = 4
index = 0

index = (index + 1) % capacity // 1
index = (index + 1) % capacity // 2
index = (index + 1) % capacity // 3
index = (index + 1) % capacity // 0

```

In questo modo:

- TAIL avanza in modo circolare durante **enqueue**
- FRONT avanza in modo circolare durante **dequeue**

Stato della coda

La struttura dati è la seguente.

L'implementazione può prevedere un array statico o un array allocato dinamicamente, come visto per lo stack.

```

typedef struct queue {
    int data[MAX_SIZE];
    int FRONT;
    int TAIL;
} Queue;

```

```

typedef struct queue {
    int *data;
    int FRONT;
    int TAIL;
    int capacity;
} Queue;

```

Nel seguito useremo l'array allocato dinamicamente.

Convenzione adottata:

- coda **vuota** → $\text{FRONT} = -1, \text{TAIL} = -1$
 - coda **piena** → $(\text{TAIL} + 1) \% \text{capacity} == \text{FRONT}$
-

Funzioni principali

`initializeQueue`

Alloca e inizializza una nuova coda vuota.

```
Queue *initializeQueue(int capacity) {
    if (capacity <= 0) {
        return NULL;
    }

    Queue *queue = malloc(sizeof(Queue));
    if (queue == NULL) {
        return NULL;
    }

    queue->data = malloc((size_t) capacity * sizeof(int));
    if (queue->data == NULL) {
        free(queue);
        return NULL;
    }

    queue->FRONT = -1;
    queue->TAIL = -1;
    queue->capacity = capacity;

    return queue;
}
```

`isEmpty`

Verifica se la coda è vuota.

La coda è vuota quando entrambi gli indici sono a -1.

```
bool isEmpty(const Queue *queue) {
    return queue == NULL || (queue->FRONT == -1 && queue->TAIL == -1);
}
```

isFull

Verifica se la coda è piena.

Nel caso di array circolare, la coda è piena quando la posizione successiva a TAIL coincide con FRONT.

```
bool isFull(const Queue *queue) {
    if (queue == NULL) {
        return false;
    }

    return (queue->TAIL + 1) % queue->capacity == queue->FRONT;
}
```

enqueue

Inserisce un elemento in fondo alla coda.

- Se la coda è vuota → inizializza FRONT e TAIL
- Altrimenti → aggiorna TAIL in modo circolare
- Inserisce il valore nella posizione TAIL

```
bool enqueue(Queue *queue, int value) {
    if (queue == NULL || isFull(queue)) {
        return false;
    }

    if (isEmpty(queue)) {
        queue->FRONT = 0;
        queue->TAIL = 0;
    } else {
        queue->TAIL = (queue->TAIL + 1) % queue->capacity;
    }

    queue->data[queue->TAIL] = value;
    return true;
}
```

dequeue

Rimuove e restituisce l'elemento in testa alla coda.

- Legge il valore in FRONT
- Se era l'unico elemento → resetta la coda
- Altrimenti → aggiorna FRONT in modo circolare

```
bool dequeue(Queue *queue, int *value) {
    if (queue == NULL || value == NULL || isEmpty(queue)) {
        return false;
    }

    *value = queue->data[queue->FRONT];

    if (queue->FRONT == queue->TAIL) {
        queue->FRONT = -1;
        queue->TAIL = -1;
    } else {
        queue->FRONT = (queue->FRONT + 1) % queue->capacity;
    }

    return true;
}
```

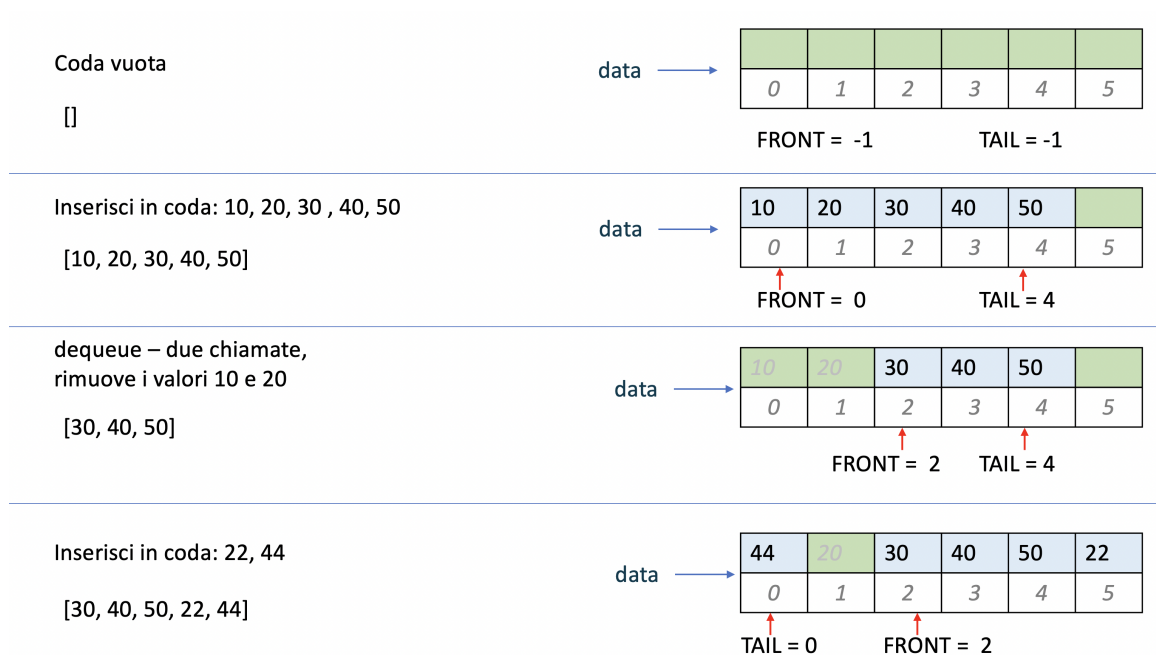
deleteQueue

Libera tutta la memoria associata alla coda.

```
void deleteQueue(Queue *queue) {
    if (queue == NULL) {
        return;
    }

    free(queue->data);
    free(queue);
}
```

Esempio



Osservazioni

- Tutte le operazioni principali (**enqueue**, **dequeue**, **peek**) hanno costo $O(1)$.
- L'uso dell'array circolare evita sprechi di memoria.
- La gestione degli indici tramite l'operatore modulo è il punto chiave dell'implementazione mediante array circolare.
- La distinzione tra stato **vuoto** e **pieno** è fondamentale e deve essere gestita con attenzione.

Note su capacity

Nelle queue implementate mediante array circolare, le condizioni di **pieno** e **vuoto** possono essere definite in modi diversi, a seconda della rappresentazione scelta.

In questa implementazione abbiamo usato:

- coda vuota:

```
FRONT == -1 && TAIL == -1
```

- coda piena:

```
(TAIL + 1) % capacity == FRONT
```

In questo caso è possibile utilizzare **tutte le celle dell'array**, e la capacità reale è esattamente `capacity`.

In altre implementazioni **non si usa uno stato speciale** (come `FRONT = -1`). Gli stati sono distinti **solo** tramite la relazione tra `FRONT` e `TAIL`.

In questa variante:

- `TAIL` rappresenta l'indice della **prossima posizione libera** (non dell'ultimo elemento inserito);
- la condizione

```
FRONT == TAIL
```

indica **coda vuota**.

Per evitare che questa stessa condizione si verifichi anche quando la coda è piena, si lascia **sempre una cella libera**. Di conseguenza, la capacità effettiva della struttura è:

```
capacity - 1
```

Implementazione della Queue mediante lista concatenata

Una queue può essere implementata anche tramite **lista concatenata**. In questo caso gli elementi non sono memorizzati in un array, ma in nodi allocati dinamicamente.

Ogni nodo contiene un valore ed un puntatore al nodo successivo.

```
typedef struct node {  
    int value;  
    struct node *next;  
} Node;
```

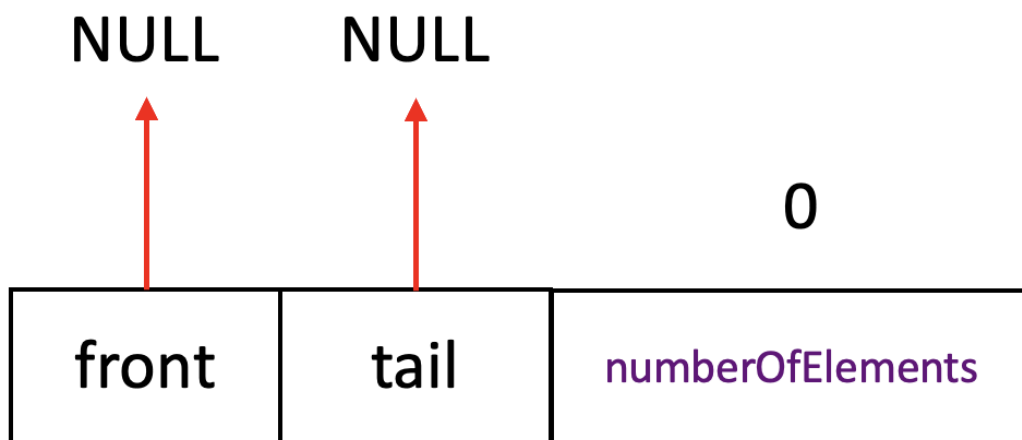
La struttura `Queue` mantiene due puntatori:

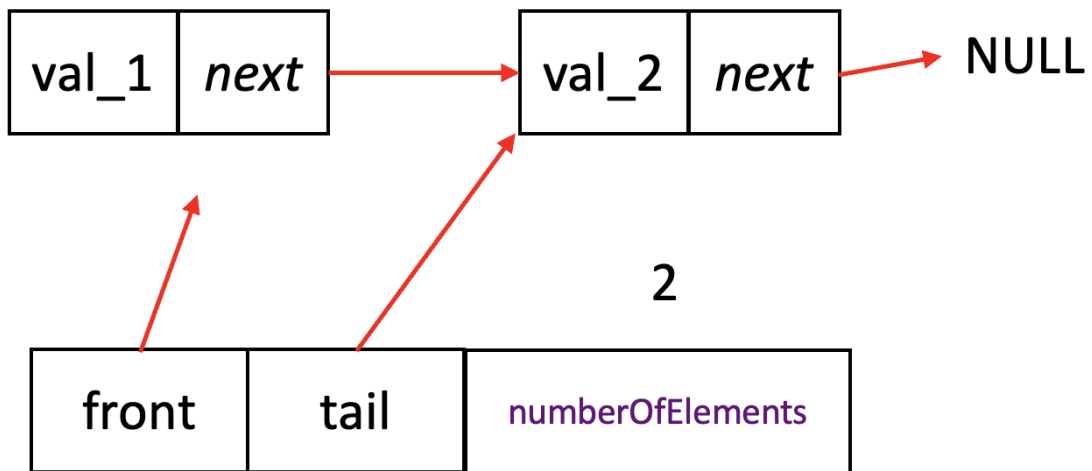
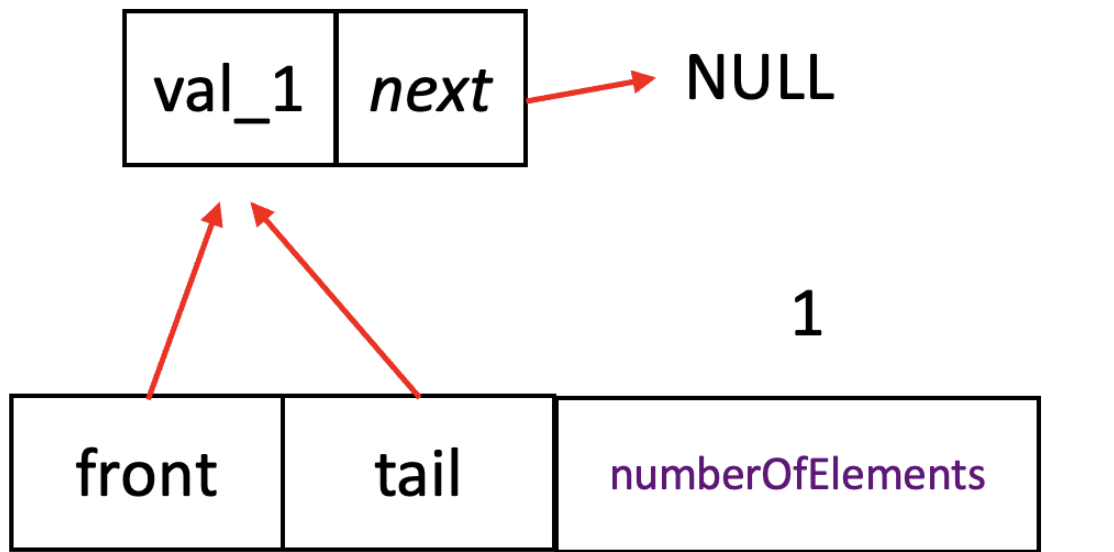
- **front**: punta al primo elemento, cioè quello che verrà rimosso dal prossimo `dequeue`;
- **tail**: punta all'ultimo elemento, cioè il punto in cui verrà inserito il prossimo elemento con `enqueue`.

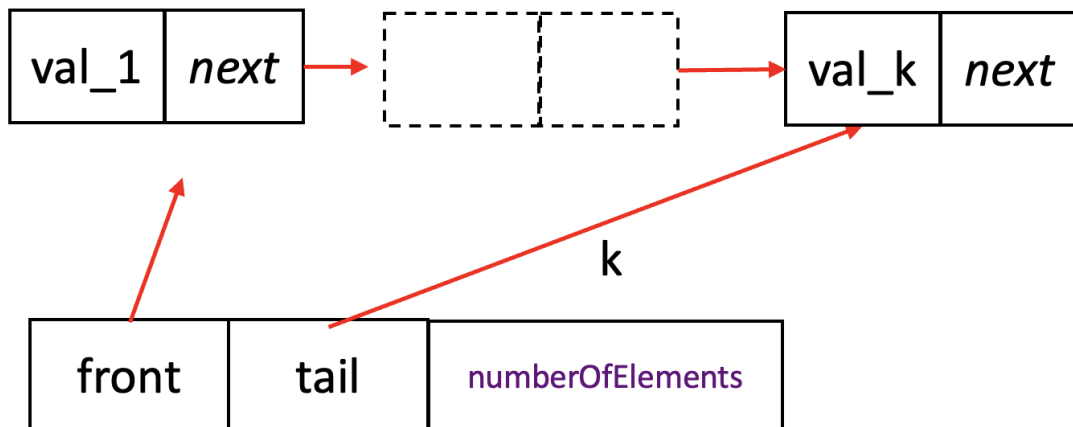
```
struct queue {  
    Node *front;  
    Node *tail;  
    int numberOfElements; // opzionale  
};
```

Questa scelta permette di inserire e rimuovere elementi in $O(1)$, senza dover spostare dati.

Esempio di coda vuota, coda con un elemento, coda con più elementi.







Interfaccia

```
Queue *initializeQueue(void);
bool isEmpty(const Queue *queue);
bool isFull(const Queue *queue);
bool enqueue(Queue *queue, int value);
bool dequeue(Queue *queue, int *value);
void deleteQueue(Queue *queue);
```

Rispetto alla queue implementata con array:

- `initializeQueue()` non riceve una capacità;
- `isFull()` restituisce sempre `false`, perché non esiste una capacità massima prefissata;
- `enqueue()` può comunque fallire se `malloc` non riesce ad allocare un nuovo nodo.

Strutture dati interne

```
typedef struct node {
    int value;
    struct node *next;
} Node;
```



```
struct queue {
    Node *front;
    Node *tail;
    int numberOfElements; // opzionale
};
```

createNewNode

Funzione privata usata per creare un nuovo nodo. Alloca memoria per un nodo, assegna il valore **val** e inizializza il puntatore **next**. Essendo dichiarata **static** e non essendo presente nell'interfaccia, è visibile solo all'interno del file **.c**.

```
static Node *createNewNode(Node *next, int val) {
    Node *newNode = malloc(sizeof(*newNode));
    if (newNode == NULL) {
        return NULL;
    }

    newNode->next = next;
    newNode->value = val;
    return newNode;
}
```

initializeQueue

Crea una nuova queue vuota. Alloca la struttura **Queue**, inizializza **front** e **tail** a **NULL** e pone il numero di elementi a zero.

```
Queue *initializeQueue(void) {
    Queue *queue = malloc(sizeof(*queue));
    if (queue == NULL) {
        return NULL;
    }

    queue->front = NULL;
    queue->tail = NULL;
    queue->numberOfElements = 0;
    return queue;
}
```

isEmpty

Verifica se la queue è vuota. La queue è vuota quando `front == NULL`. Per robustezza, viene considerata vuota anche una queue `NULL`.

```
bool isEmpty(const Queue *queue) {  
    return queue == NULL || queue->front == NULL;  
}
```

isFull

Nella versione con lista concatenata non esiste una capacità massima fissata in anticipo. Per questo motivo `isFull()` restituisce sempre `false`. L'unico limite reale è la memoria disponibile.

```
bool isFull(const Queue *queue) {  
    return false;  
}
```

enqueue

Inserisce un nuovo elemento in fondo alla queue. La funzione crea un nuovo nodo e lo collega alla fine della lista.

Occorre considerare due casi distinti:

- se la queue è vuota, il nuovo nodo diventa sia `front` sia `tail`;
- altrimenti, viene collegato dopo l'attuale `tail`, poi `tail` viene aggiornato.

```
bool enqueue(Queue *queue, int value) {  
    if (queue == NULL) {  
        return false;  
    }  
  
    Node *newNode = createNewNode(NULL, value);  
    if (newNode == NULL) {  
        return false;  
    }  
}
```

```

    }

    queue->numberOfElements += 1;

    if (isEmpty(queue)) {
        queue->front = newNode;
        queue->tail = newNode;
    } else {
        queue->tail->next = newNode;
        queue->tail = newNode;
    }

    return true;
}

```

Costo: $O(1)$.

dequeue

Rimuove l'elemento in testa alla queue. La funzione salva il valore contenuto nel nodo **front**, poi rimuove quel nodo dalla lista.

Occorre considerare due casi distinti:

- se la queue contiene un solo elemento, dopo la rimozione sia **front** sia **tail** diventano NULL;
- altrimenti, **front** viene spostato al nodo successivo.

```

bool dequeue(Queue *queue, int *value) {
    if (queue == NULL || value == NULL || isEmpty(queue)) {
        return false;
    }

    *value = queue->front->value;
    Node *p_tmp = queue->front;

    if (queue->tail == queue->front) {
        queue->front = NULL;
        queue->tail = NULL;
    } else {
        queue->front = queue->front->next;
    }
}

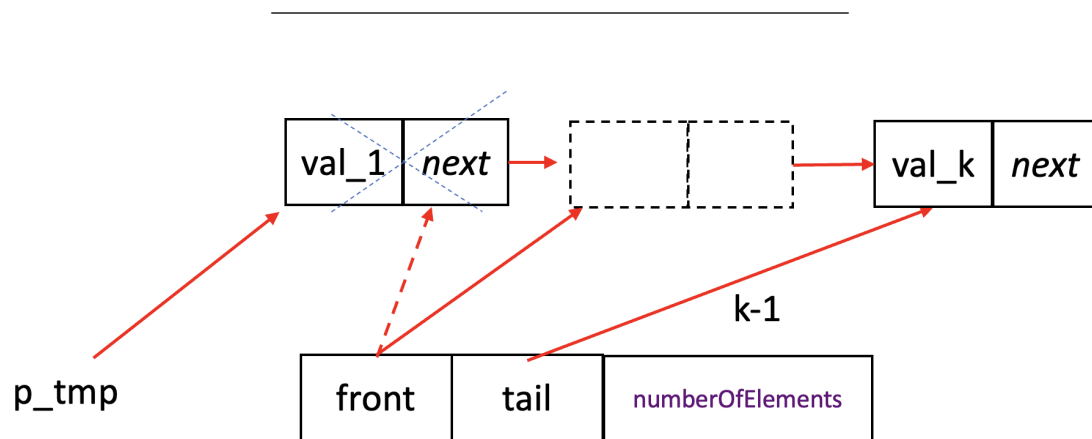
```

```

    free(p_tmp);
    queue->numberOfElements -= 1;
    return true;
}

```

Costo: $O(1)$.



```

q->front = q->front->next;

```

deleteQueue

Libera tutta la memoria associata alla queue.

La funzione attraversa la lista a partire da `front`, libera ogni nodo e infine libera la struttura `Queue`.

```

void deleteQueue(Queue *queue) {
    if (queue == NULL) {
        return;
    }

    while (!isEmpty(queue)) {
        Node *current = queue->front;
        queue->front = current->next;
        free(current);
    }
}

```

```
    free(queue);  
}
```

Funzione di debug: show

Questa funzione non fa parte dell'ADT queue. È utile solo per controllare il contenuto della struttura durante il debug.

In una versione finale dovrebbe essere rimossa dall'interfaccia e dichiarata `static` nel file `.c`.

```
void show(const Queue *queue) {  
    if (queue == NULL) {  
        printf("Queue: <null>\n");  
        return;  
    }  
  
    printf("Queue: ");  
    const Node *current = queue->front;  
    while (current != NULL) {  
        printf("%d ", current->value);  
        current = current->next;  
    }  
    printf("\n");  
}
```

Vantaggi e svantaggi della linked list

Vantaggi

- Non richiede una capacità massima fissata in anticipo.
- Non serve gestire un array circolare.
- `enqueue` e `dequeue` hanno costo $O(1)$.
- La memoria cresce dinamicamente in base al numero di elementi.

Svantaggi

- Ogni inserimento richiede una `malloc`, ogni rimozione richiede una `free`.
- Ogni nodo occupa memoria aggiuntiva per il puntatore `next`.
- Gli elementi non sono contigui in memoria, quindi l'accesso può essere meno efficiente rispetto a un array.
- La gestione della memoria è più delicata.

Considerazioni finali

I tipi di dato astratti sono particolarmente interessanti perché permettono di manipolare **entità del dominio applicativo**, invece che dettagli di basso livello legati all'implementazione. È quindi opportuno lavorare sempre al **più alto livello possibile di astrazione**.

Invece di pensare di inserire un nodo in una lista concatenata, si può ragionare in termini di operazioni sul dominio: aggiungere una cella a un foglio di calcolo, un nuovo tipo di finestra a un sistema di interfaccia grafica, oppure un vagone a una simulazione di treno. In questi casi, gli ADT possono essere modellati come `spreadsheet`, `windowSystem`, `trainSimulator`, ecc.

La domanda da porsi è: “Cosa rappresenta questo stack, lista o queue?”

Se uno stack rappresenta un insieme di dipendenti, va trattato come un insieme di dipendenti, non come uno stack. Se una lista rappresenta un insieme di fatture, va trattata come fatture, non come una lista. Se una queue rappresenta celle di un foglio di calcolo, va vista come una collezione di celle, non come una struttura generica.

In altre parole, la struttura dati è un **mezzo**, non il fine: il codice dovrebbe riflettere il **significato** dei dati più che la loro rappresentazione.

Esercizi proposti

Alcuni esercizi proposti per lo stack possono essere applicati anche alle queue, per esempio il calcolo della media degli elementi.

Altri tipi di ADT che potete provare ad implementare

→ **Deque (Double-ended queue)**

Inserimento e rimozione sia dalla testa sia dalla coda

→ **Lista circolare**

→ **Stack o Queue su array, ma con capacità dinamica**

Definire una funzione `resize()` che espanda automaticamente l'array quando pieno.

→ **time slots, numeri complessi, segmenti, angoli, matrici, ecc**

FONTI PRINCIPALI

Sedgewick, Capitolo 4, Tipi di dati astratti

Code Complete / Steve McConnell.–2nd ed.

Un estratto del Cap. 6 è disponibile insieme alle slides.¹

¹“Il riassunto, la citazione o la riproduzione di brani o di parti di opera e la loro comunicazione al pubblico sono liberi se effettuati per uso di critica o di discussione, nei limiti giustificati da tali fini e purché non costituiscano concorrenza all'utilizzazione economica dell'opera; se effettuati a fini di insegnamento o di ricerca scientifica l'utilizzo deve inoltre avvenire per finalità illustrative e per fini non commerciali.” - <https://www.normattiva.it/uri-res/N2Ls?urn:nir:stato:legge:1941-04-22;633~art70>